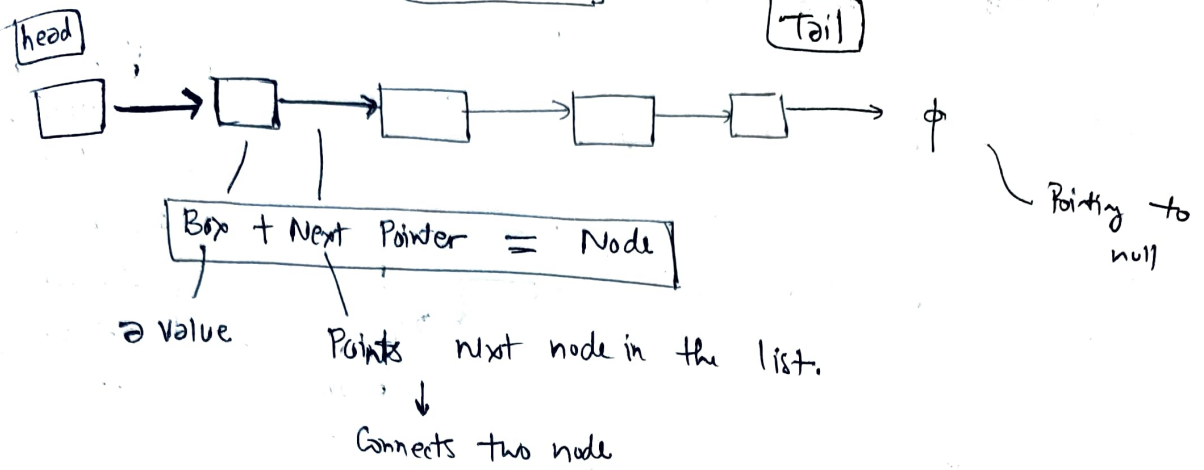


LINKED LIST

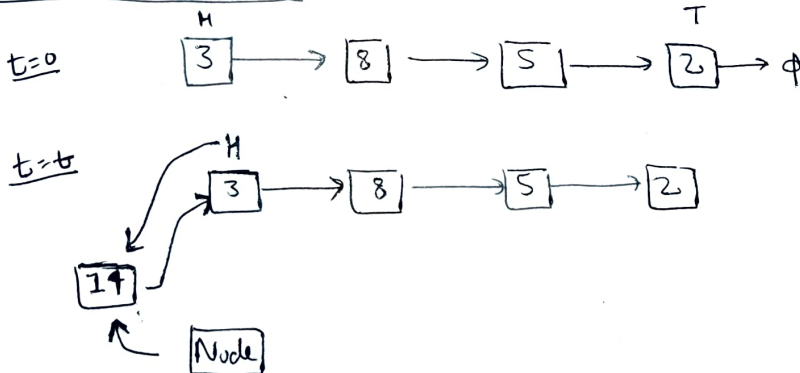


- head** $\rightarrow$ head is a reference to the first node in LL.
 $\hookrightarrow$ tells from which list starts.
- tail** $\rightarrow$ reference to the last node in LL.

Insertion in LL & Deletion in LL

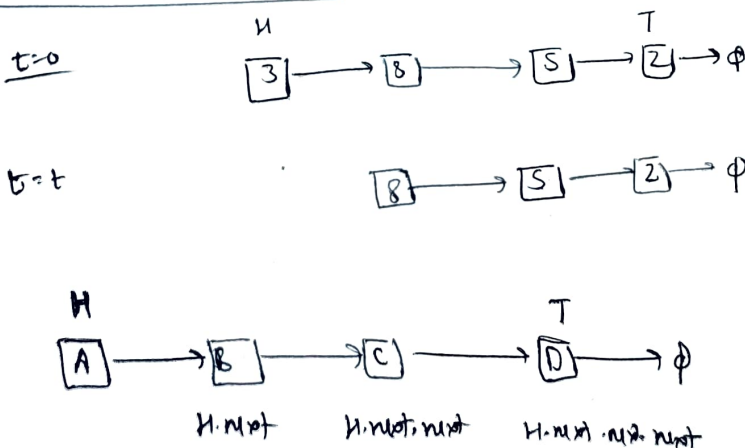
- S-1** Create new Node.
- S-2** Manage Pointers.
- S-3** Case of failure or Base Case.

① Insert at 1st Position



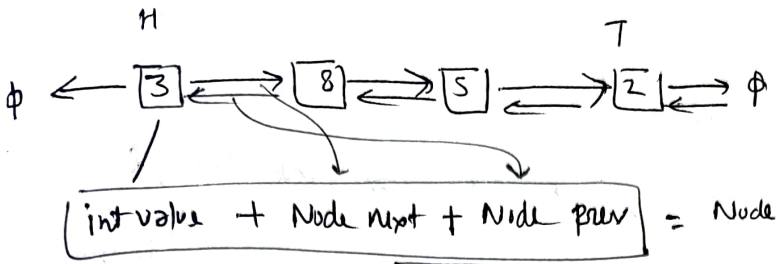
```
Node node = new Node(val);
node.next = head;
head = node;
if (tail == null) {
    tail = head;
}
size++;
```

② deletion at 1st Position



```
int val = head.value;
head = head.next;
if (head == null) {
    tail = null;
}
size--;
return val;
```

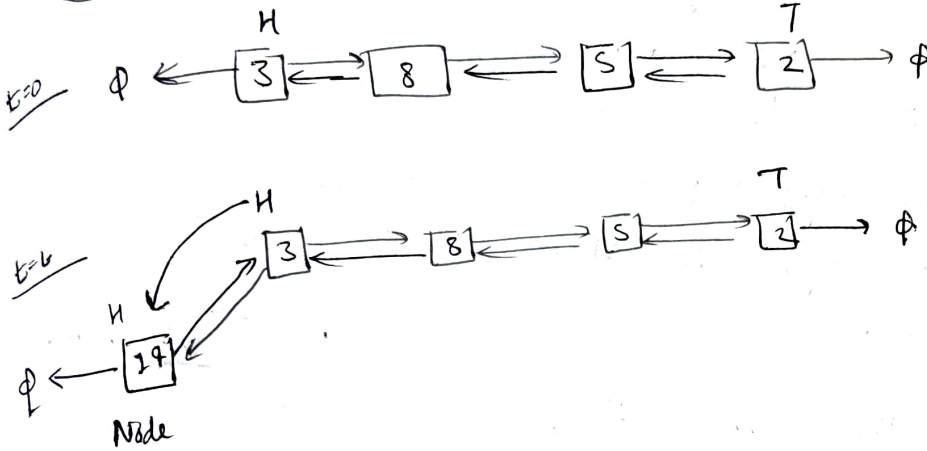
Doubly LL



int value + Node next + Node prev = Node

INSERTION

① At 1st Index



Node node = new Node(val);

node.next = head;

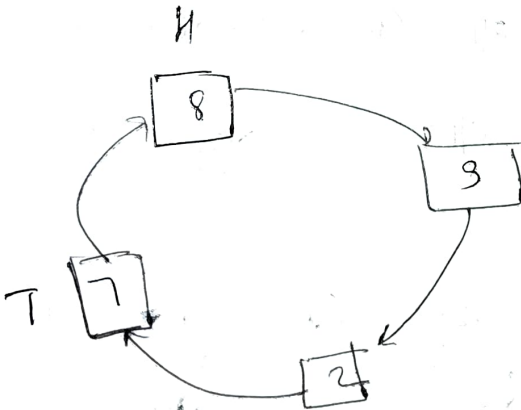
node.prev = null;

if (head != null) {

head.prev = node;

head = node;

CLL Circular Linked List



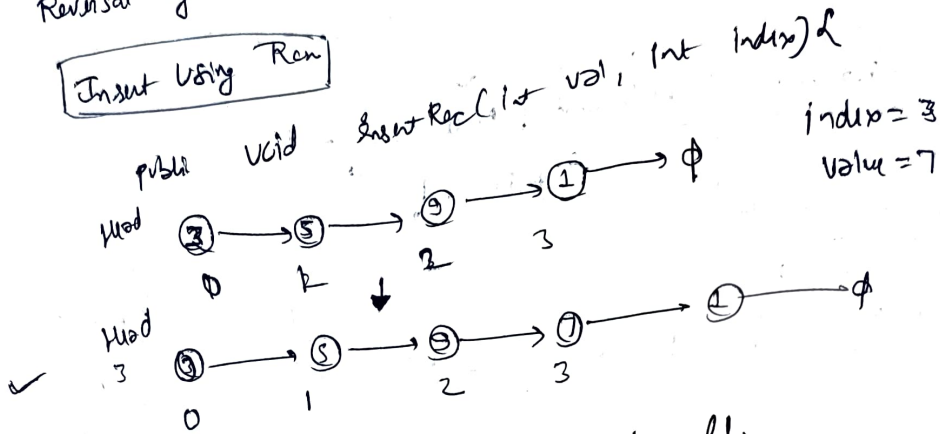
Class Node {
int val;
Node next;
}

No null pointer —
until list is empty

At 1st

- ① 2 Pointers
- ② Reversal of LL

Insert Using Rec



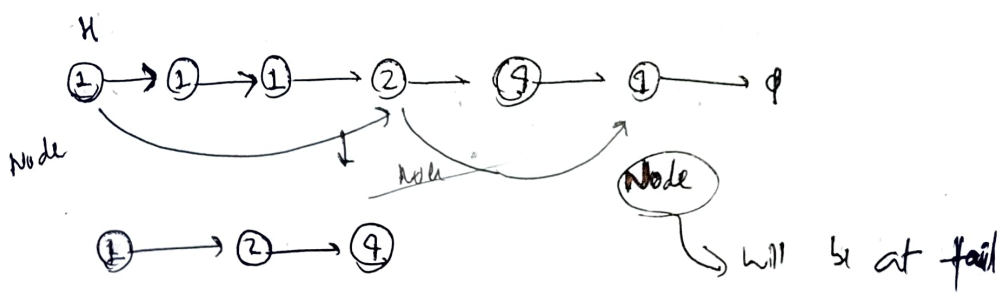
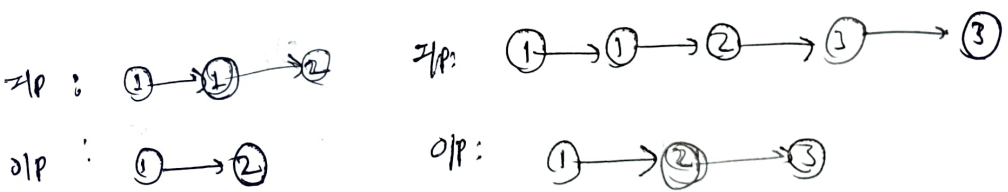
- ① void return type + make changes to ll.
- ② Node return type that return the list node to change the structure.

index, value



How LL should look like?

Remove Duplicates



node.val = value skip it

public void duplicates {
Node node = head;

while (node.next != null) {
if (node.val == node.next.val)

Merge two sorted Lists

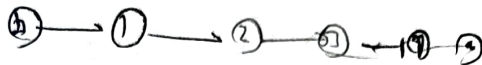
list 1



list 2



or



Compare element by element



One of the ll finished traversing

S-1 compare h1 & h2

S-2 if h1 < h2 move h1

if h2 < h1 move h2

S-3 if any one become empty add it to the remaining list

if (node == null) return null;

Start from head

listNode current = node;

Node head f = first head;

Node head s = second head;

LL ans = new LL();

while (f != null & s != null) {

if (f.val < s.val) {

ans.insertLast(f.val);

}

f = f.next;

}

else {

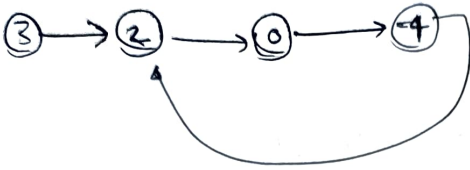
s = s.next;

}

Naive

Linked List Cycle

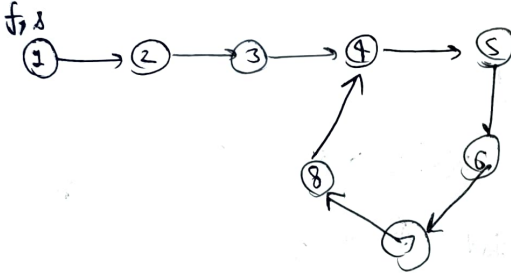
2/1



Input: head = [3, 2, 0, -4], pos = 2

Fast & Slow Pointer Method

- Cycle detection
- Finding a node in cycle, etc.

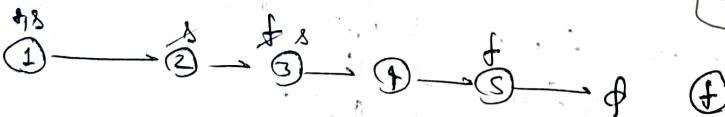
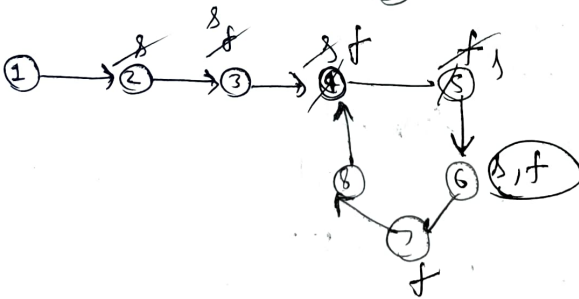
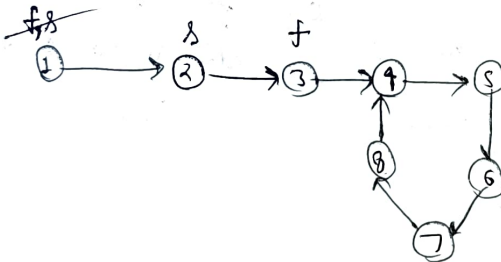


With every iteration

f $\rightarrow$ 2 step

s $\rightarrow$ 1 step

Iteration

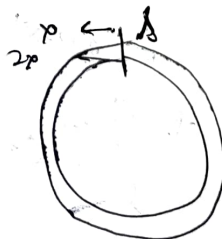


Meeting at the Node

Cycle is that

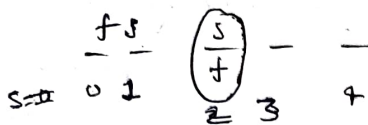
Why?

fasten will take over & meet slower one at a time.



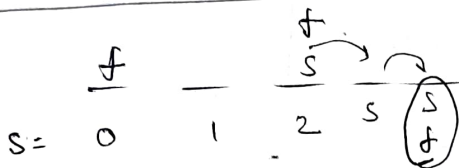
Possibilities

① fast is 1 step behind



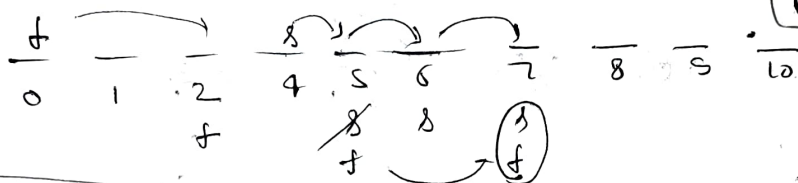
Must in 1 step

② fast is 2 step behind slow



Must in 2 step

③ fast is 3 step behind



Must in 3 step

TC = $O(N)$ → not running as

Class ListNode
int val;

ListNode next;

ListNode (int);

};

ListNode head;

to avoid
run time
err

fast.next → check current node exists
fast.next → there is a node after fast.

public boolean hasCycle (ListNode head) {

 ListNode fast = head;

 ListNode slow = head;

 while (fast != null && fast.next != null) {

 fast = fast.next.next;

← two step move fast

 slow = slow.next;

← 1 step move slow

 if (fast == slow) {

 return true;

 }
 return false;

Also find Length of Cycle

keep fast step + move slow until $f \neq s$

if (fast == slow) {

Let Node temp = slow;

let length = 0;

do {

temp = temp->next;

length++;

3

while (temp != slow);

return true;

length of Cycle

return if f == 8

Let Node temp = slow;

let length = 0;

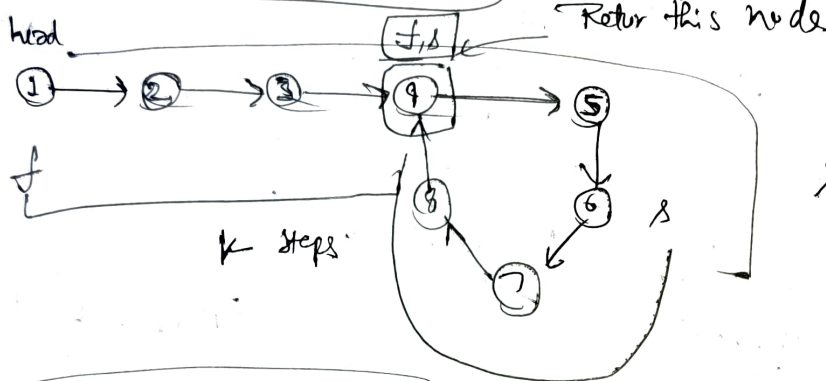
do {

temp = temp->next;

do

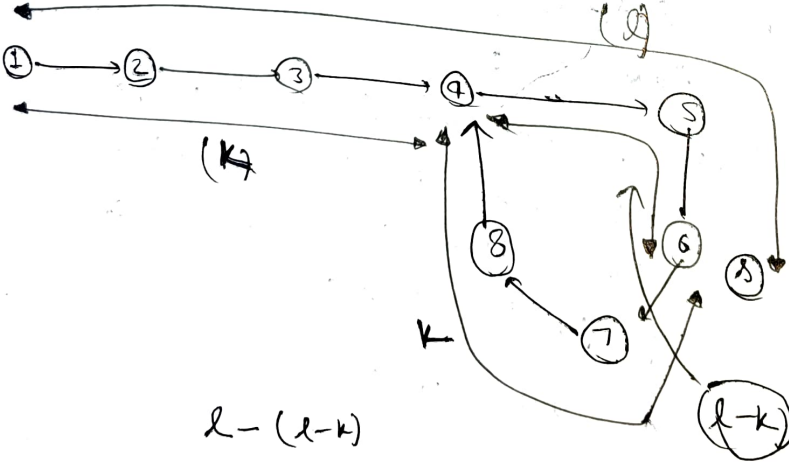
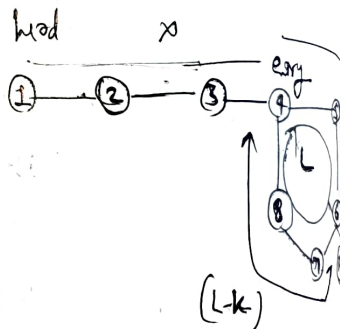
let length++;

Starting Point of Cycle



Where do to put

second pointer such that $k = k$.



$L - (L - k)$

$\Rightarrow L - L + k$

$\Rightarrow k$

$k = L - k$

(1)

① Find length of Cycle

② Move s ahead by length of cycle k

③ Move s + 1 one by one it will meet at start

Happy Number

$$12 = 1^2 + 2^2 = 5$$

$$5^2 = 25 = 2^2 + 5^2 = 29$$

$$29 = 2^2 + 9^2 = 85$$

$$85 = 8^2 + 5^2 = 145$$

$$145 = 1^2 + 4^2 + 5^2 = 72$$

$$72 = 7^2 + 2^2 = 53$$

$$53 = 5^2 + 3^2 = 34$$

$$94^2 = 20$$

$$2^2 + 0^2 = 4$$

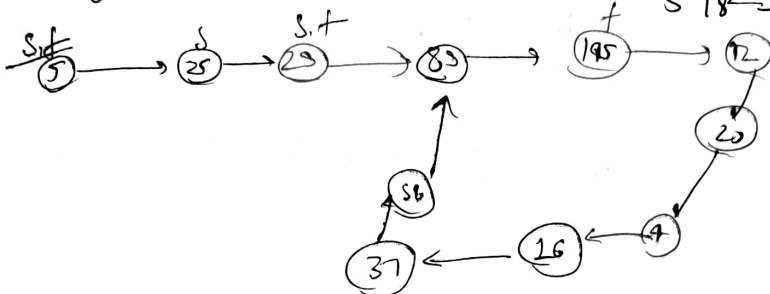
$$9^2 = 16$$

$$1^2 + 6^2 = 37$$

$$3^2 + 7^2 = 9 + 49 = 58$$

$$58 = 5^2 + 8^2 = 89$$

$$8^2 + 9^2 = 145$$



Imagine as cycle of LL

public boolean isHappy (int n)

int slow = n;

int fast = n;

```
private int findSquare (int n) {
    int ans = 0;
    while (n > 0) {
        int rem = n % 10;
        ans += rem * rem;
        n = n / 10;
    }
    return ans;
}
```

588

n/.10

while (num > 0)

{

int rem = num % 10;

ans += rem * rem;

num = num / 10;

}

do {

slow = findSquare (slow);

fast = findSquare (findSquare (fast));

} while (slow != fast);

if (slow == 1) {

return true;

} return false;

Middle of the LL



slow & fast pointer method

while fast travel or at end.

slow will be at middle.

public ListNode middleNode (ListNode head)

ListNode f = head;

ListNode s = head;

while (f != null & f.next != null)

s = s.next;

f = f.next;

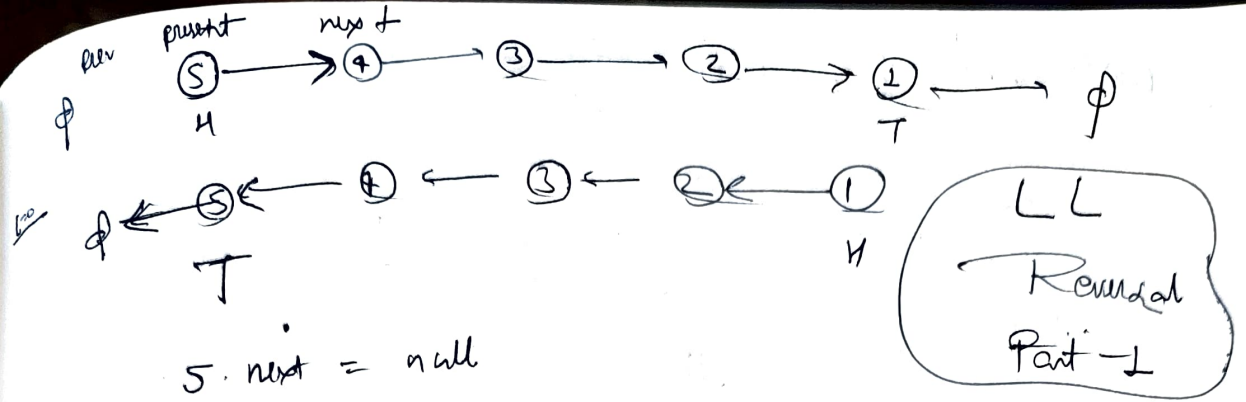
3

3

return s;

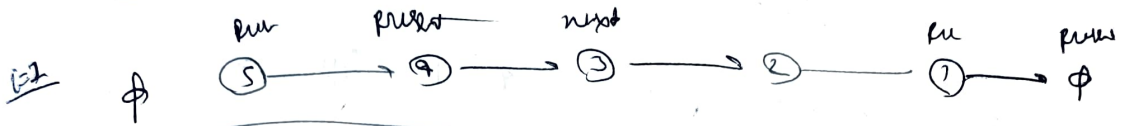
Sort List





5. next = null

prev next = prev



prev = prev

prev = next

next = next.next

Null pointer check

In the end prev = last element

```
public void reverse ( ) {
    if (size < 2) {
        return;
    }
```

Node prev = null;

Node temp = head;

Node next = temp.next;

while (temp != null) {

temp.next = prev;

prev = temp;

temp = next;

if (next != null) {

next = next.next;

}

}

head = prev;

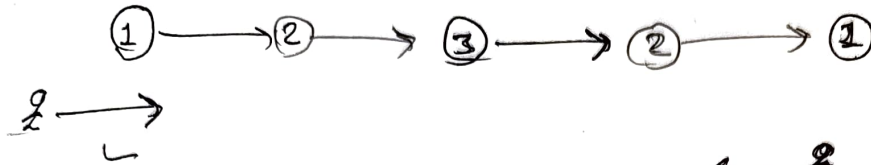
do inside the loop to avoid null pointer except;

V-IMP

Palindrome LL

Ques

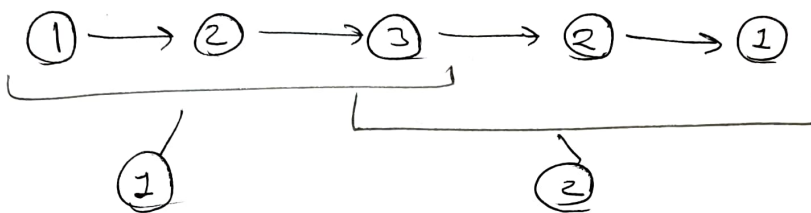
had



Read from $L =$ Read from R

1 2 3 2 1 = 1 2 3 2 1

or



L is reverse of R .

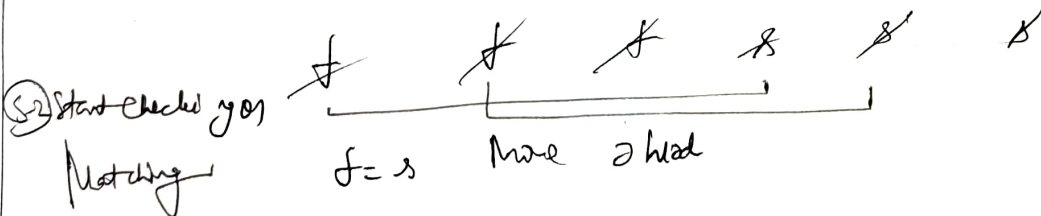
S-1 Find Mid



1 → 2 → 3 → 3 → 2 → 1

S-1 Reverse 1/2 LL

1 → 2 → 3 → 2 → 1 → 2 → 3

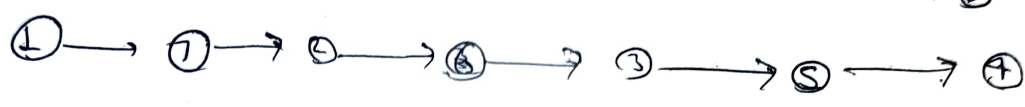
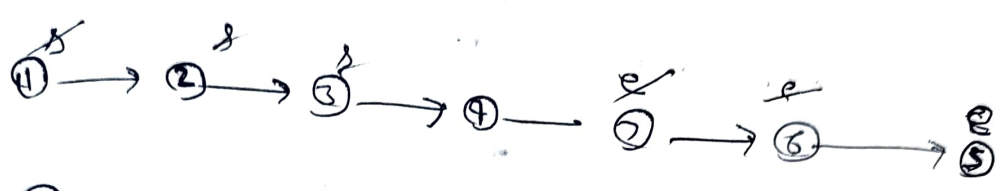
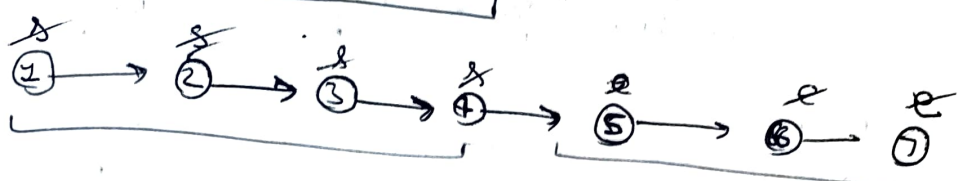


Empty 'return'.

S-3

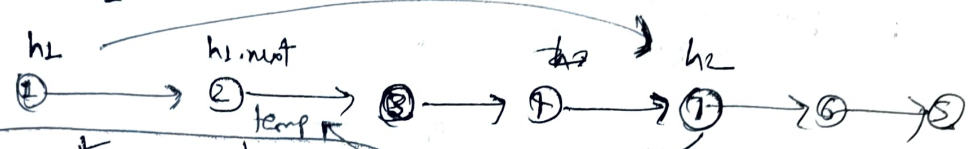
to reverse the second 1/2.

Reorder LL



or

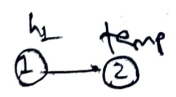
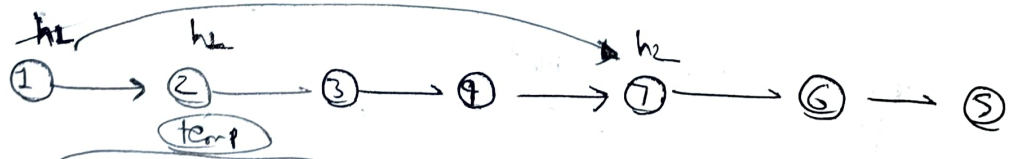
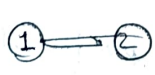
temp = h1
 h1.next = h2
 h1 = h2



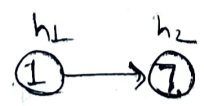
S1

temp = h1.next
 h1.next = h2 ✓
~~h1 = h1.next~~ temp
 temp = h2.next
 h2.next = h1
 h2 = temp

h1 = temp



temp = h1.next
 h1.next = h2
 h1 = temp
~~temp = h1.next~~



temp = h2.next
 h2.next = h1
 h2 = temp

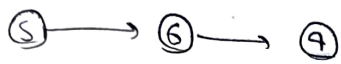
if (head == null || head.next == null)
 {
 return;
 }

ListNode mid = middleNode(head);

ListNode h2 = reverseList(mid);

ListNode h1 = head;

Ques 1



Add 2
LL



```
ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    if (l1.val + l2.val > 10) ListNode node;
    int rem = while (l1 != null || l2 != null) {
        sum = l1.val + l2.val
```

```
while (sum < 10) while (sum < 10)
for (i=0; i<10; i++) Node node = sum; num = num * 10;
```

```
ListNode dummyHead = new ListNode(0);
```

```
ListNode current = dummyHead;
int carry = 0;
```

```
while (l1 != null || l2 != null || carry != 0) {
```

```
int val1 = (l1 != null) ? l1.val : 0;
int val2 = (l2 != null) ? l2.val : 0;
int sum = val1 + val2 + carry;
int carry = sum / 10;
```

Temporary operator

Condition ? T : F

S1 Ek dummy Node banao.

S2 int val1 = (l1 != null) ? l1.val : 0;

val2 =

int sum = val1 + val2 + carry

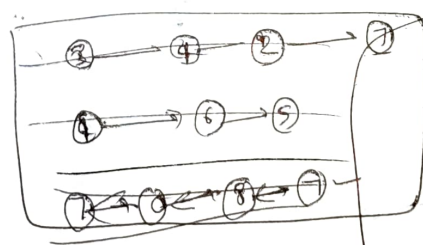
carry = sum / 10

current = new ListNode (sum % 10)

current = current

Create a node
then join the
list 1 to list 2

int val1 = (l1 != null) ? l1.val : 0;
int sum = val1 + val2 + carry;
carry = sum / 10;



Create a reverse for
private ListNode reverse (ListNode head)

```
ListNode prev = null;
while (head != null) {
```

```
ListNode next = head.next;
```

```
head.next = prev;
```

```
prev = head;
```

```
head = next;
```

```
}
return prev;
```

3

Ques 2

